

Dino Game Bot

Brief Overview

Explainer document generated from the project source

What this document is

A concise, accurate overview of `dino-game-bot`. For full detail (every function, both diagrams' surrounding explanation, the full verification log) see `deep-dive.pdf` in the same folder. License: PolyForm Strict 1.0.0 (source viewable, no reuse or redistribution rights); not MIT.

1 What it does

Plays Chrome's offline Dino game (`chrome://dino/`) by capturing screen pixels in a loop and pressing space when an obstacle enters a trigger zone just ahead of the dino. It never touches the game's DOM or internal state, only what it can see on screen, driven through Selenium (a browser-automation library) for the keypress and `ImageGrab.grab()` (a screenshot-to-image function) for the vision side.

2 Detection logic

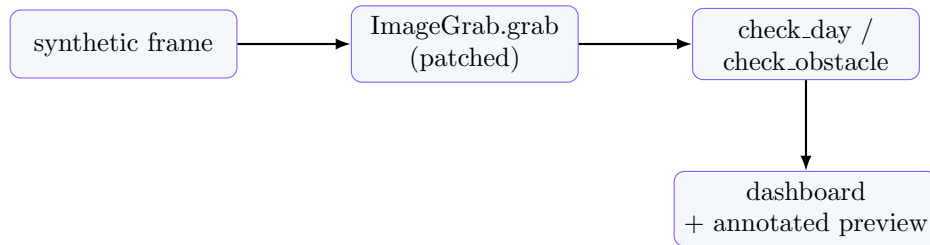
`check_day()` grabs a wide screen region and counts white vs. dark-grey pixels to decide day or night mode. `check_obstacle()` grabs a narrower region right in front of the dino and checks for the *opposite* mode's color (obstacles render dark on a light day background, and light on a dark night background); if found, it means an obstacle is in the trigger zone and the bot jumps immediately. There is no distance estimate, just presence-in-box.

3 Status dashboard

A small `tkinter` window (or a throttled console printout if no display is available) shows the current mode, whether an obstacle is detected, a running jump count, elapsed time, and a rolling log of recent events, refreshed once per loop iteration without blocking it.

4 New: demo mode

There is no live Chrome game to actually play in every environment this project runs in. `python3 main.py --demo` replays six hand-drawn PNG frames (`demo_frames/`, built by `tools/generate_demo_frames.py` using the real colors and coordinates from `main.py`) through the *unmodified* `check_day/check_obstacle` functions, by temporarily swapping what `ImageGrab.grab()` returns instead of adding a branch inside the detection code. The dashboard gains an image panel (`DemoTkStatusView`, a new subclass, live mode's dashboard class is untouched) showing each frame with the sampled boxes drawn on top: blue for the day/night box, green or red for the obstacle box depending on the result.



Real run, all six frames matched expectations: day/night correctly identified every time, jump triggered only on the two frames with an obstacle drawn inside the trigger box (not the two with an obstacle drawn further away, "approaching but not yet in range").

5 New: an optional, proven robustness option

The original obstacle check needs a pixel to be an *exact* RGB match, which is fragile against small color drift a real screen capture can have (anti-aliasing, dithering, compression) that a flat synthetic frame never shows on its own. Both detection functions gained an optional `tolerance` parameter, default 0 (identical to the old exact-match behavior, unchanged everywhere it is currently called). `tools/compare_detection_robustness.py` proves this with real numbers, not a claim: it builds a "noisy" copy of every demo frame (a deterministic ± 6 -per-channel checkerboard shift, so no pixel is ever exactly the original color) and compares the old exact-match logic against tolerance-based matching on both sets.

	clean frames	noisy frames
old (tolerance=0)	6/6 correct	4/6 correct
new (tolerance=8)	6/6 correct	6/6 correct

No regression on clean frames, full recovery on noisy frames; the option is available to callers but not turned on by default, since it has only been tested against synthetic noise, not a real screen capture.

6 Verified vs. not

Verified here, with real commands and real output: frame generation (6 files), demo mode against a live X display and again with the display unset (headless console fallback), the robustness comparison script's before/after numbers, and that `ImageGrab.grab` and `build_status_view()`'s default behavior are unaffected after a demo run.

Not verified, needs a real machine with Chrome: actually launching Chrome and playing the real game, whether the hardcoded screen coordinates match any given real display, and whether the tolerance option helps against real capture noise rather than the synthetic approximation used here.